I-Logical Approaches Knowledge Bases, Entailment and Contractions of Tensors

Foundations of Neuro-Symbolic AI

Alex Goessmann

University of Applied Science Würzburg-Schweinfurt (THWS)

March 23, 2026

Outline

- Knowledge Bases
- Tensor Networks
- Contractions
- Entailment
- Python package `tnreason`

Motivation: Curse of dimensionality

Observation

The number of coordinates of a propositional formula increases exponentially with the number d of variables.

Its not just a depiction problem - it is a fundamental intractability in storage and processing!

Curse of dimensionality: Storage problem

When reasoning about a factored system with many variables (i.e. d is large), the naive enumeration of all models to formulas is infeasible.

Motivation: Knowledge bases

There are further sparsity principles:

- We often have a conjunction over multiple formulas.
- Each formula is just affected by a few variables.

Definition (Knowledge base)

Knowledge bases are conjunctions over formulas $\{f_\ell \,:\, \ell \in [p]\}$, that is

$$\mathcal{KB} = \bigwedge_{\ell \in [p]} f_\ell \,.$$

We represent them by a tensor

$$\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = \begin{cases} 1 & \text{if} \quad \forall \ell \in [p] : f_\ell \left[X_{[d]} = x_{[d]}\right] = 1 \\ 0 & \text{else}\,. \end{cases}$$

How to exploit this sparsity for more effective representation?

Outline

- Knowledge Bases
- Tensor Networks
- Contractions
- Entailment
- Python package `tnreason`

Combination of tensors with distinct variables: Tensor products

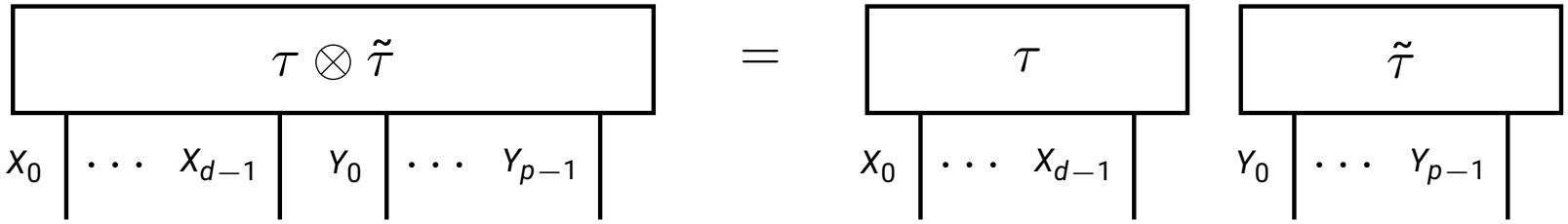
Definition (Tensor Product)

Let there be two tensors $\tau\left[X_{[d]}\right]$ and $\tilde{\tau}\left[Y_{[p]}\right]$ with disjoint variables. Then there tensor product is a tensor

$$(\tau \otimes \tilde{\tau})\left[X_{[d]}, Y_{[p]}\right]$$

defined for $x_{[d]} \in \bigtimes_{k\in[d]}[m_k]$ and $y_{[p]} \in \bigtimes_{\ell\in[p]}[n_\ell]$ as

$$(\tau \otimes \tilde{\tau})\left[X_{[d]} = x_{[d]}, Y_{[p]} = y_{[p]}\right] = \tau\left[X_{[d]} = x_{[d]}\right] \cdot \tilde{\tau}\left[Y_{[p]} = y_{[p]}\right] .$$



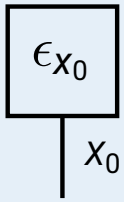
Example: One-hot encodings

Definition (One-hot encoding to atomic representations)

The one-hot encoding of the state $x_0 \in [m_0]$ of a variable X_0 with dimension $m_0 \in \mathbb{N}$ is the map

$$\epsilon_{x_0}[X_0 = y_0] = \begin{cases} 1 & \text{if} \quad x_0 = y_0 \\ 0 & \text{else} \end{cases}$$

that is the vector in $\mathbb{R}^{m_0}$ with a 1 at the position of the state and 0 else.



$$= \begin{array}{c} \begin{pmatrix} 0 & \cdots & 0 & 1 & 0 & \cdots & 0 \end{pmatrix} [X_0] \\ \begin{array}{ccccccc} \vdots & & & \vdots & & & \vdots \\ 0 & & & x_0 & & & m_0 - 1 \end{array} \end{array}$$

Example: The one-hot encoding $\epsilon_2[X_0] \in \mathbb{R}^4$ has the coordinates

$$\epsilon_2[X_0] = \begin{pmatrix} 0 & 0 & 1 & 0 \end{pmatrix} [X_0]$$

Example: One-hot encodings

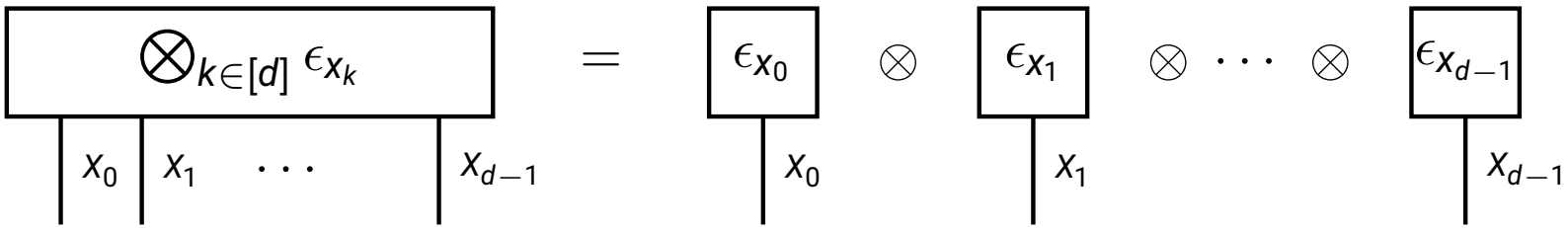
Definition (One-hot encoding to factored representations)

The one-hot encoding of a factored representation with variables $X_{[d]} = \{X_k \,:\, k \in [d]\}$ in state $x_0, \ldots, x_{d-1}$ is the tensor product

$$\epsilon_{x_{[d]}}\left[X_{[d]}\right] = \bigotimes_{k \in [d]} \epsilon_{x_k}\left[X_k\right]$$

of the one-hot encodings of the variables.

We can depict the one-hot encoding by



Example: One-hot encodings

Example: The one-hot encoding $\epsilon_{(0,1,1)}\left[X_0, X_1, X_2\right] \in \bigotimes_{k\in[3]} \mathbb{R}^4$ with the coordinates

$$\epsilon_{(0,1,1)}\left[X_0 = x_0, X_1 = x_1, X_2 = x_2\right] = \begin{cases} 1 & \text{if} \quad x_0 = 0,\ x_1 = 1 \text{ and } x_2 = 1 \\ 0 & \text{else} \end{cases}$$

is the tensor product of

$$\epsilon_0\left[X_0 = x_0\right] = \begin{cases} 1 & \text{if} \quad x_0 = 0 \\ 0 & \text{else} \end{cases} \quad , \quad \epsilon_1\left[X_1 = x_1\right] = \begin{cases} 1 & \text{if} \quad x_1 = 1 \\ 0 & \text{else} \end{cases}$$

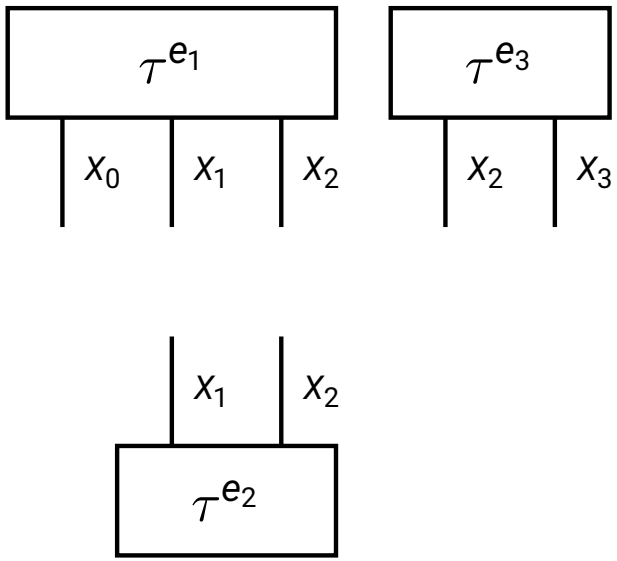
and

$$\epsilon_1\left[X_2 = x_2\right] = \begin{cases} 1 & \text{if} \quad x_2 = 1 \\ 0 & \text{else} \end{cases} .$$

Tensor networks

Tensor networks are build by

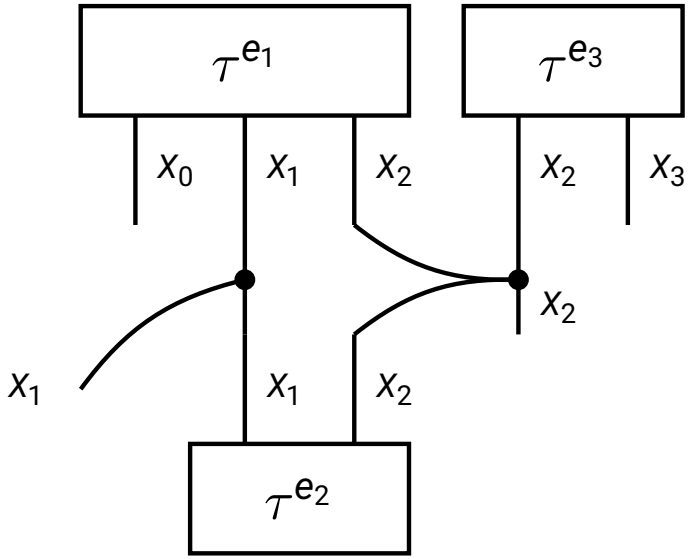
- Separated combination of tensors $\tau^{e_1}, \tau^{e_2}, \tau^{e_3}$: Tensor Products



Tensor networks

Tensor networks are build by

- Separated combination of tensors $\tau^{e_1}, \tau^{e_2}, \tau^{e_3}$: Tensor Products
- Connecting Tensors along common legs: Contractions (with all legs left open)



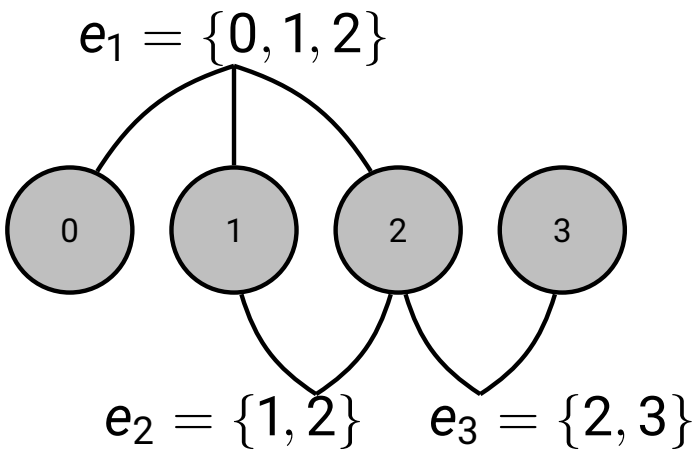
Depicting common legs: Hypergraphs

Definition (Hypergraph)

A hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ consists of two sets:

- Nodes $v \in \mathcal{V}$ (corresponding with variables)
- Hyperedges $e \in \mathcal{E}$, where each is is a subset $e \subset \mathcal{V}$ of nodes (corresponding with tensors)

We depict nodes by gray circles and hyperedges as connections of them:



In this example:

- $\mathcal{V} = \{0, 1, 2, 3\}$
- $\mathcal{E} = \{\{0, 1, 2\}, \{1, 2\}, \{2, 3\}\}$

Associating tensors with hyperedges

Let us use the hypergraph formalism to depict tensor networks

- Nodes are decorated by the categorical variables on the tensor legs:

$$\{0, \ldots, d-1\} \quad \text{represent the variables} \quad \{X_0, \ldots, X_{d-1}\}$$

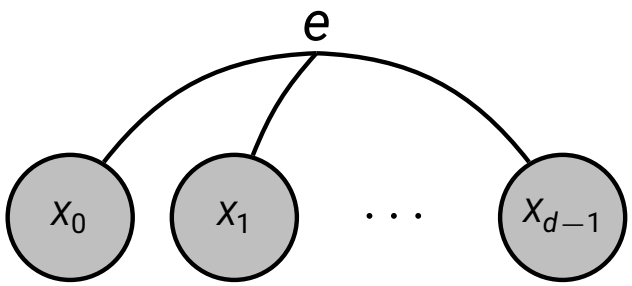
  Further, we associate the dimensions m_v of the respective variable with them.

- Hyperedges are decorated by tensors

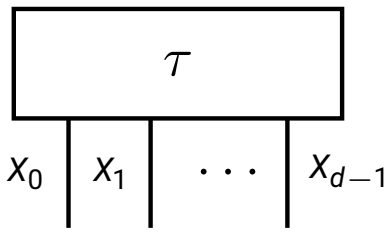
$$e = \{v \,:\, v \in e\} \quad \text{represents the tensor} \quad \tau^e\,[X_e] \in \bigotimes_{v \in e} \mathbb{R}^{m_v}$$

We now decorate hyperedges by tensors and depict them by a) hypergraph diagrams b) tensor network diagrams:

a)



b)



Tensor Networks

Definition (Tensor Network)

Let $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ be a hypergraph with nodes decorated by variables X_v with dimensions $m_v \in \mathbb{N}$. and hyperedges $e \in \mathcal{E}$ decorated by core tensors

$$\tau^e [X_e] \in \bigotimes_{v \in e} \mathbb{R}^{m_v} .$$

Then the set $\{\tau^e [X_e] \,:\, e \in \mathcal{E}\}$ builds the tensor network to the decorated hypergraph $\mathcal{G}$ and we say it decomposes the tensor

$$\tau^{\mathcal{G}} [X_{\mathcal{V}}] := \langle \tau^e [X_e] \,:\, e \in \mathcal{E} \rangle_{[X_{\mathcal{V}}]} .$$

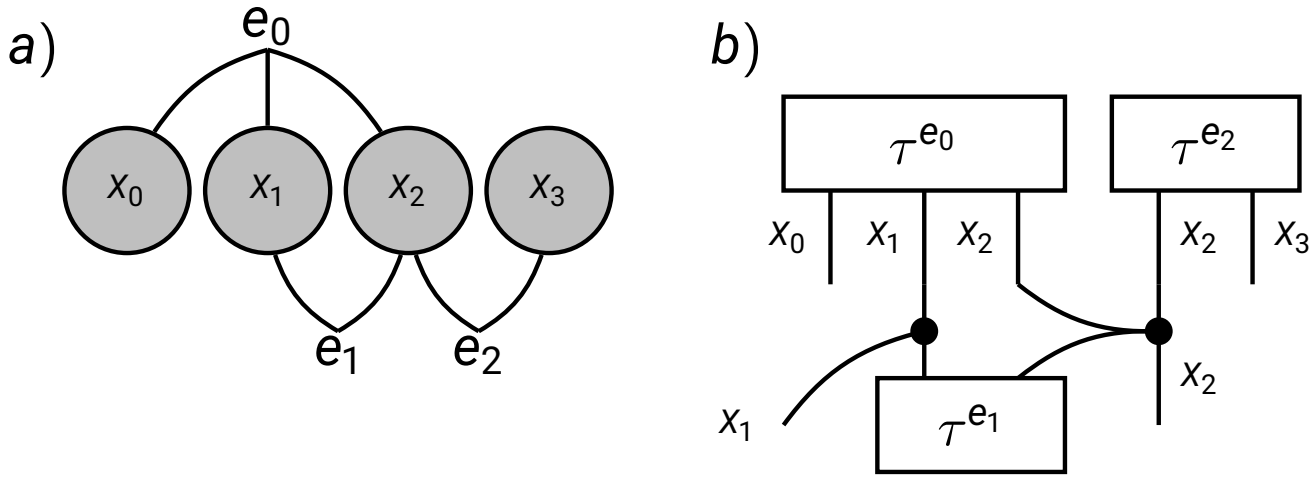
with coordinates to $x_{\mathcal{V}} \in \bigtimes_{v \in \mathcal{V}} [m_v]$ by

$$\tau^{\mathcal{G}} [X_{\mathcal{V}} = x_{\mathcal{V}}] = \prod_{e \in \mathcal{E}} \tau^e [X_e = x_e] .$$

Graphical Representation of Tensor Networks

Example of a tensor network:

- Four categorical variables $\mathcal{V} = \{X_0, X_1, X_2, X_3\}$ appearing as nodes and decorated by their dimensions $m_{:}$.
- Hyperedges $e_0 = \{X_0, X_1, X_2\}$, $e_1 = \{X_1, X_2\}$ and $e_2 = \{X_2, X_3\}$ decorated with tensors



We use two diagrammatic depictions:

a) The defining hypergraph
b) The tensor network diagram

Example of a Tensor Network: Hadamard Product of vectors

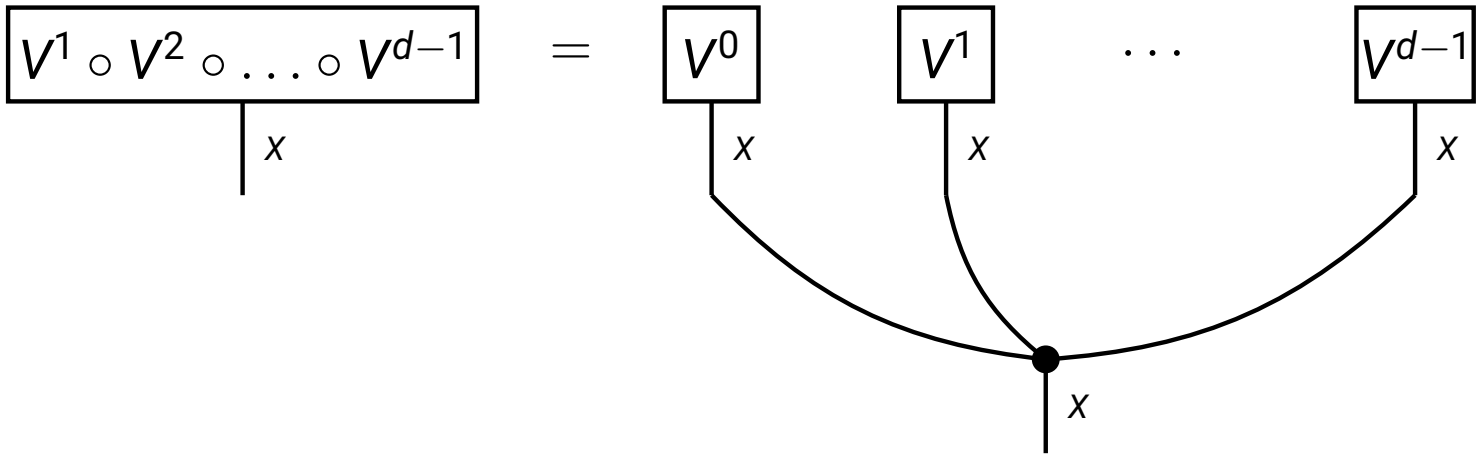
Let $V^k[X] \in \mathbb{R}^m$ be vectors for $k \in [d]$. Their Hadamard product is the vector

$$(V^0 \circ V^1 \circ \ldots \circ V^{d-1})[X] \in \mathbb{R}^m$$

defined by the coordinates at $x \in [m]$ by

$$\left(V^0 \circ V^1 \circ \ldots \circ V^{d-1}\right)[X = x] = \prod_{k \in [d]} V^k[X = x]\,.$$

We visualize the product by leg connection of multiple vectors:



Curse of dimensionality of factored representations

Tensor spaces have a problem with exponential increase of dimensions as

$$\dim\left[\bigotimes_{v\in\mathcal{V}}\mathbb{R}^{m_v}\right]=\prod_{v\in\mathcal{V}}m_v\,.$$

Tensor Networks can break this curse of dimensionality by storing the decorating tensors instead

$$\sum_{e\in\mathcal{E}}\dim\left[\bigotimes_{v\in e}\mathbb{R}^{m_v}\right]=\sum_{e\in\mathcal{E}}\left(\prod_{v\in e}m_v\right)$$

Representation strategy

Store the recipe (that is the core tensors), not the result (the contracted tensor network).

Knowledge bases as tensor networks

Given a knowledge base $\mathcal{KB} = \{f_\ell \,:\, \ell \in [p]\}$ we

- Create for each $\ell \in [p]$ a hyperedge e_ℓ containing the nodes to the variables X_{e_ℓ} appearing in the formula f_ℓ
- Build a set of nodes, which correspond with the variables appearing in the formulas

$$\mathcal{V} = \bigcup_{\ell \in [p]} e_\ell$$

Theorem (Tensor network decomposition of knowledge bases)

For any knowledge base $\mathcal{KB} = \{f_\ell \,:\, \ell \in [p]\}$ the corresponding tensor $\mathcal{KB}\,[X_\mathcal{V}]$ is decomposed into the formulas by

$$\mathcal{KB}\,[X_\mathcal{V}] = \langle \{ f_\ell\,[X_{e_\ell}] \,:\, \ell \in [p] \} \rangle_{[X_\mathcal{V}]} \,.$$

Knowledge bases as tensor networks

Theorem (Tensor network decomposition of knowledge bases)

For any knowledge base $\mathcal{KB} = \{f_\ell \,:\, \ell \in [p]\}$ the corresponding tensor $\mathcal{KB}\,[X_\mathcal{V}]$ is decomposed into the formulas by

$$\mathcal{KB}\,[X_\mathcal{V}] = \langle \{ f_\ell\,[X_{\mathsf{e}_\ell}] \,:\, \ell \in [p] \} \rangle_{[X_\mathcal{V}]} \,.$$

Proof: For each $x_\mathcal{V} \in \bigtimes_{v \in \mathcal{V}} [2]$ we have $\mathcal{KB}\,[X_\mathcal{V} = x_\mathcal{V}] = 1$ if and only if for all $\ell \in [p]$ $f_\ell\,[X_{\mathsf{e}_\ell} = x_{\mathsf{e}_\ell}] = 1$. This is equal to

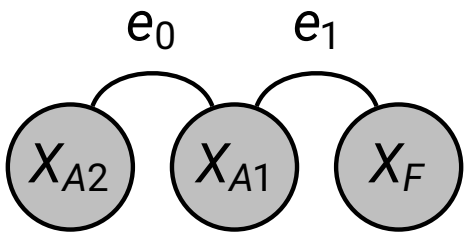
$$\mathcal{KB}\,[X_\mathcal{V} = x_\mathcal{V}] = \prod_{\ell \in [p]} f_\ell\,[X_{\mathsf{e}_\ell} = x_{\mathsf{e}_\ell}] \,.$$

Tensor network decomposition of the toy accounting system

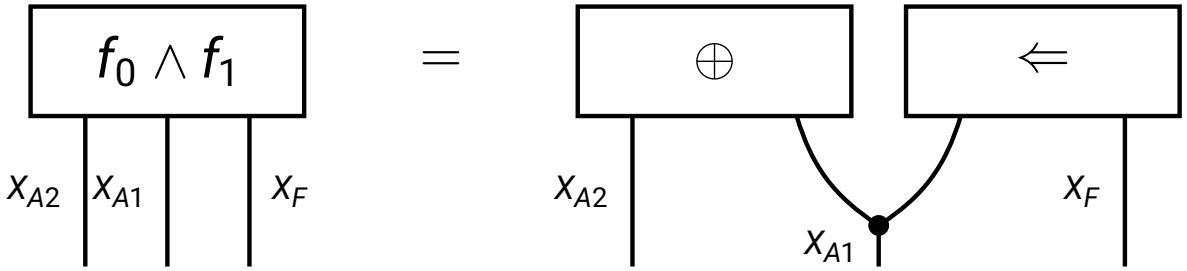
Toy accounting system:

- $f_0 = (X_{A1} \oplus X_{A2})$ by $e_0 = \{A1, A2\}$
- $f_1 = (X_F \Rightarrow X_{A1})$ by $e_1 = \{F, A1\}$

The corresponding hypergraph is thus:



We have a tensor network decomposition by decorating tensors as:



Outline

- Knowledge Bases
- Tensor Networks
- Contractions
- Entailment
- Python package `tnreason`

Scalar products of vectors

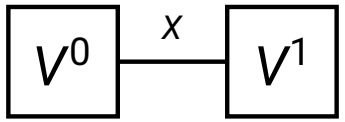
Let there be a variable X taking values in $[m]$ and vectors

$$V^0[X] \in \mathbb{R}^m \quad \text{and} \quad V^1[X] \in \mathbb{R}^m \,.$$

We denote their scalar product by

$$\left\langle V^0[X], V^1[X] \right\rangle_{[\varnothing]} = \sum_{x \in [m]} V^0[X = x] \cdot V^1[X = x] \,.$$

and visualize it diagramatically with closed legs:



Einstein sum convention

The indices to variables, which not left open in an contraction, are summed over.

Matrix-vector contraction

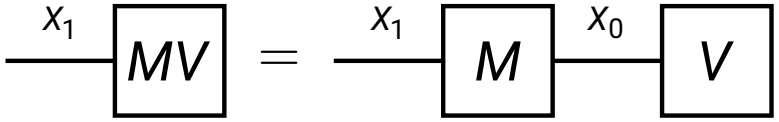
Let there now be a matrix $M[X_1, X_0] \in \mathbb{R}^{m_1 \times m_0}$ and a vector $V[X_0] \in \mathbb{R}^{m_0}$. The matrix-vector (product) contraction is the vector

$$\langle M[X_1, X_0], V[X_0] \rangle_{[X_1]}$$

with coordinates to $x_1 \in [m_1]$ by

$$\langle M[X_1, X_0], V[X_0] \rangle_{[X_1 = x_1]} = \sum_{x_0 \in [m_0]} M[X_1 = x_1, X_0 = x_0] \cdot V[X_0 = x_0] \,.$$

We visualize it diagrammatically with a closed and one open leg:



Example: Matrix-vector contractions for reasoning

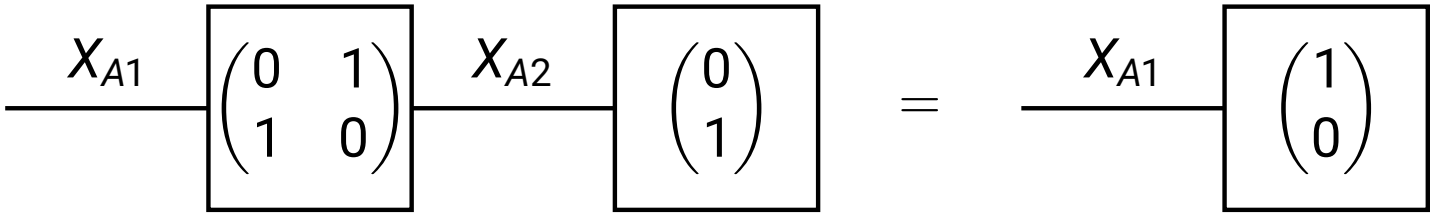
Example:

Knowing that $X_{A2} = 1$, what are the possible values of X_{A1}?

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} [X_{A1}, X_{A2}] \begin{pmatrix} 0 \\ 1 \end{pmatrix} [X_{A2}] = \begin{pmatrix} 0 \cdot 0 + 1 \cdot 1 \\ 1 \cdot 0 + 0 \cdot 1 \end{pmatrix} [X_{A1}] = \begin{pmatrix} 1 \\ 0 \end{pmatrix} [X_{A1}]$$

Interpretation: When X_{A2} is known to be True, then X_{A1} is False.

In the blockwise notation the matrix-vector multiplication is depicted by:



Contraction of Tensor Networks

Definition (Contraction)

Let $\tau^{\mathcal{G}}$ be a tensor network on a decorated hypergraph $\mathcal{G} = (\mathcal{V}, \mathcal{E})$. For any subset $\mathcal{U} \subset \mathcal{V}$ we define the contraction to be the tensor

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}]} \in \bigotimes_{v \in \mathcal{U}} \mathbb{R}^{m_v}$$

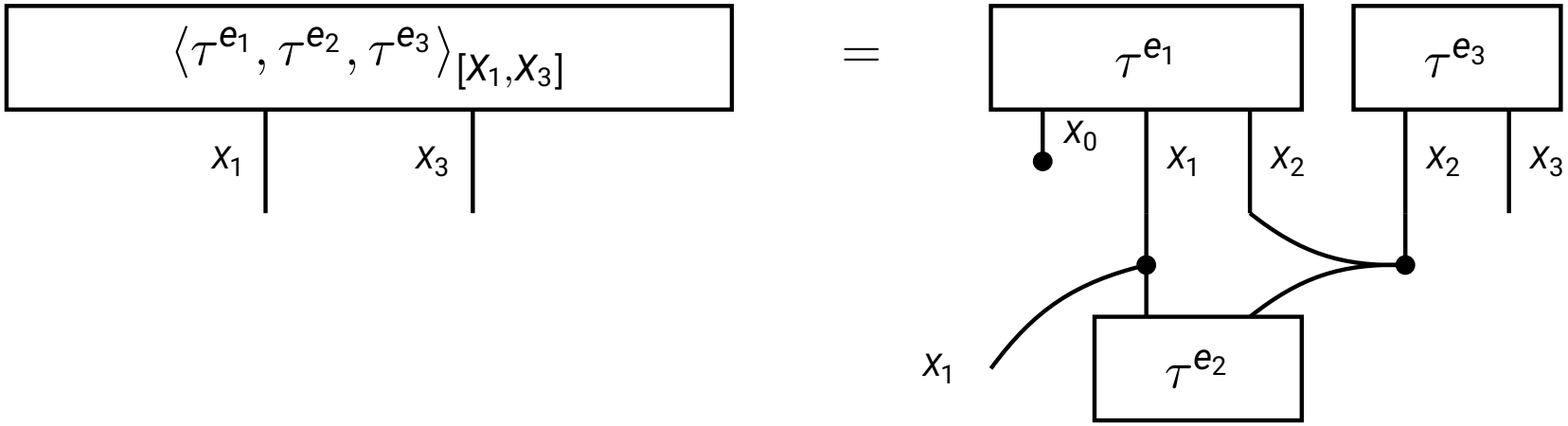
defined coordinatewise by the sum

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}} = x_{\mathcal{U}}]} = \sum_{x_{\mathcal{V} \setminus \mathcal{U}} \in \times_{v \in \mathcal{V} \setminus \mathcal{U}} [m_v]} \prod_{e \in \mathcal{E}} \tau^{e} \left[X_e = x_e \right] .$$

Contractions with closed variables

Example of a contraction:

- Four categorical variables $\mathcal{V} = \{X_0, X_1, X_2, X_3\}$ appearing as nodes and decorated by their dimensions $m_{:}$.
- Hyperedges $e_0 = \{X_0, X_1, X_2\}$, $e_1 = \{X_1, X_2\}$ and $e_2 = \{X_2, X_3\}$ decorated with tensors
- Contraction leaving $\{X_1, X_3\}$ open, by summing over the states of $\{X_0, X_2\}$



Outline

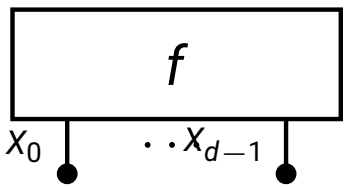
- Knowledge Bases
- Tensor Networks
- Contractions
- Entailment
- Python package `tnreason`

Contraction without open variables: Model counts

So far: Knowledge bases are contractions of formulas keeping all variables open.

Now: Contractions without open variables:

$$\langle f\left[X_{[d]}\right]\rangle_{[\varnothing]} = \sum_{x_{[d]}\in[2]^d} f\left[X_{[d]} = x_{[d]}\right] = \#\{x_{[d]} : f\left[X_{[d]} = x_{[d]}\right] = 1\}$$

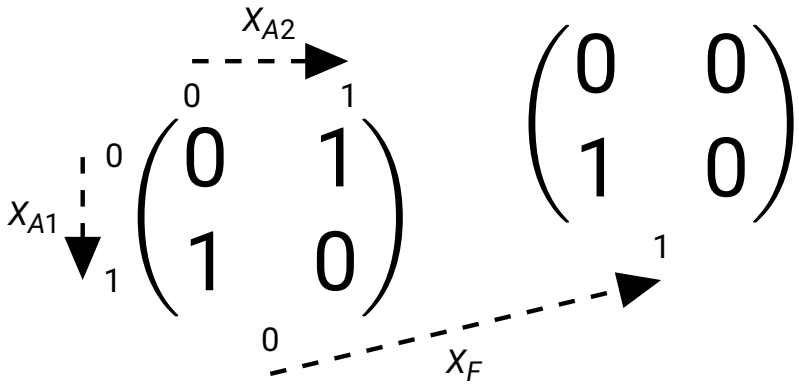


Contraction of formulas without open variables

The contraction $\langle f\left[X_{[d]}\right]\rangle_{[\varnothing]}$ of a formula without leaving variables open is the count of its models (that are states $x_{[d]}$ with $f\left[X_{[d]} = x_{[d]}\right] = 1$). If $\langle f\left[X_{[d]}\right]\rangle_{[\varnothing]} = 0$, we say the formula is unsatisfiable.

Example: Model count in the toy accounting system

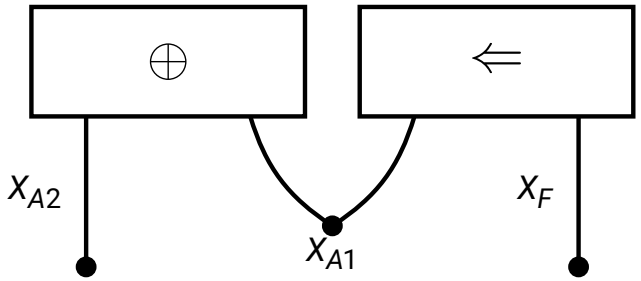
Recall the coordinate-wise representation of the toy accounting system:



$$\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix}$$

The model count of the toy accounting system is thus:

$$\langle f_0\left[X_{A1}, X_{A2}\right], f_1\left[X_F, X_{A1}\right]\rangle_{[\varnothing]} =$$



$$= 3$$

Entailment

Definition (Entailment)

Let $\mathcal{KB}\left[X_{[d]}\right]$ and $f\left[X_{[d]}\right]$ be two propositional formulas. If for all worlds $x_{[d]}$, for which $\mathcal{KB}\left[X_{[d]} = x_{[d]}\right] = 1$, we also have $f\left[X_{[d]} = x_{[d]}\right] = 1$, then we say that f is entailed by $\mathcal{KB}$ and denote

$$\mathcal{KB} \models f\,.$$

How to express this as a contraction?

For any propositional formulas we have $\mathcal{KB} \models f$ if and only if

$$\left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \left\langle \mathcal{KB}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}$$

Vanishing contraction criterion

Since $\mathcal{KB} \models f$ is equal to

$$\left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = \left\langle \mathcal{KB}\left[X_{[d]}\right]\right\rangle_{[\varnothing]}$$

it is further equal to

$$\begin{aligned} 0 &= \left\langle \mathcal{KB}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} - \left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} \\ &= \left\langle \mathcal{KB}\left[X_{[d]}\right], \mathbb{I}\left[X_{[d]}\right]\right\rangle_{[\varnothing]} - \left\langle \mathcal{KB}\left[X_{[d]}\right], f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} \\ &= \left\langle \mathcal{KB}\left[X_{[d]}\right], \mathbb{I}\left[X_{[d]}\right] - f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} \\ &= \left\langle \mathcal{KB}\left[X_{[d]}\right], \neg f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} . \end{aligned}$$

Contraction criterion for entailment

For any propositional formulas we have $\mathcal{KB} \models f$ if and only if

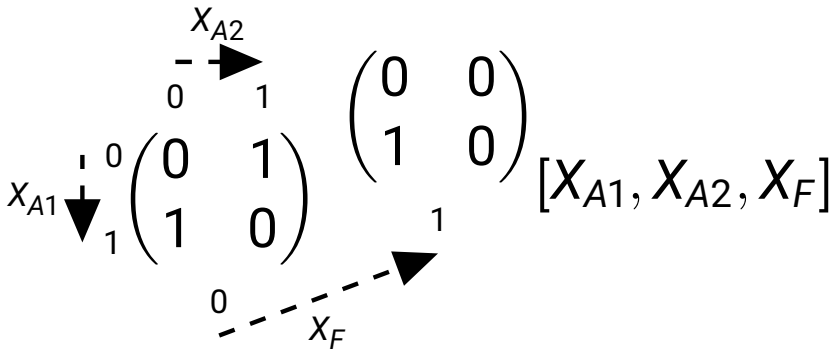
$$\left\langle \mathcal{KB}\left[X_{[d]}\right], \neg f\left[X_{[d]}\right]\right\rangle_{[\varnothing]} = 0 ,$$

that is when $\mathcal{KB} \wedge (\neg f)$ is unsatisfiable.

Entailment in the toy accounting system

Recall: The toy accounting system $\mathcal{KB}\,[X_{A1}, X_{A2}, X_F]$ is represented by the tensor

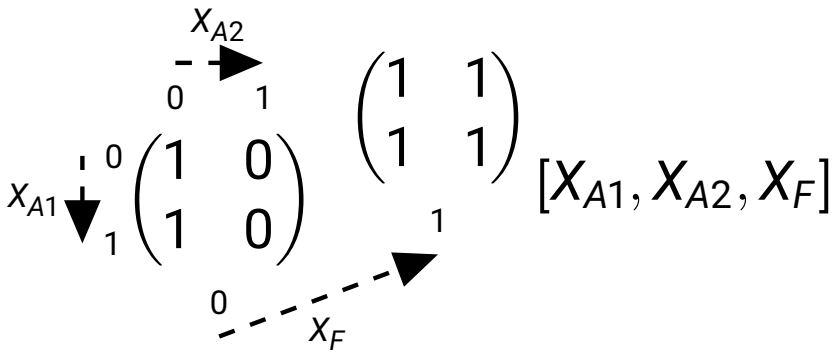
$$(X_{A1} \oplus X_{A2}) \wedge (X_F \Rightarrow X_{A1}) \;=$$



$$[X_{A1}, X_{A2}, X_F]$$

Question: Is the formula $f\,[X_{A1}, X_{A2}, X_F] = (X_{A2} \Rightarrow \neg X_F)$ entailed?

$$(X_{A2} \Rightarrow \neg X_F) \;=$$

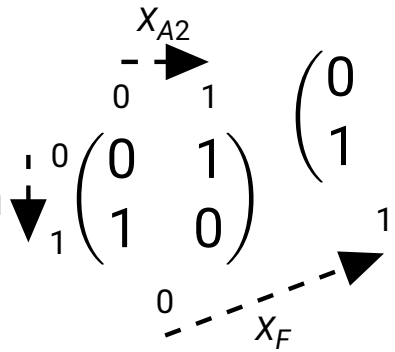


$$[X_{A1}, X_{A2}, X_F]$$

Entailment in the toy accounting system

Recall: The toy accounting system $\mathcal{KB}\,[X_{A1}, X_{A2}, X_F]$ is represented by the tensor

$$(X_{A1} \oplus X_{A2}) \wedge (X_F \Rightarrow X_{A1}) \;=$$

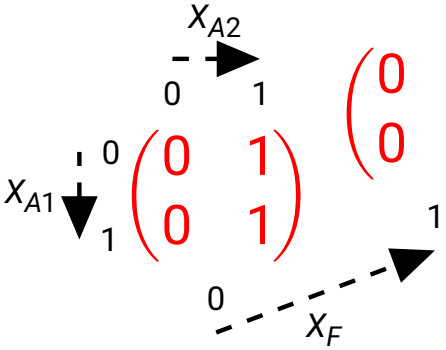


$$\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} [X_{A1}, X_{A2}, X_F]$$

Question: Is the formula $f\,[X_{A1}, X_{A2}, X_F] = (X_{A2} \Rightarrow \neg X_F)$ entailed?

1. Negate the formula f

$$\neg(X_{A2} \Rightarrow \neg X_F) \;=$$



$$\begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} [X_{A1}, X_{A2}, X_F]$$

Entailment in the toy accounting system

Recall: The toy accounting system $\mathcal{KB}\,[X_{A1}, X_{A2}, X_F]$ is represented by the tensor

$$(X_{A1} \oplus X_{A2}) \wedge (X_F \Rightarrow X_{A1}) \;=\; \begin{array}{c} \\ x_{A1} \\ \\ \end{array}\!\!\begin{array}{c} x_{A2} \\ \begin{array}{cc} 0 & 1 \end{array} \\ \begin{array}{c} 0 \\ 1 \end{array}\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} \\ \begin{array}{cc} 0 & 1 \end{array} \\ x_F \end{array}\!\!\begin{array}{c} \\ \\ \\ \end{array} [X_{A1}, X_{A2}, X_F]$$

$$\langle \mathcal{KB}, \neg f \rangle_{[\varnothing]} = 1$$

Question: Is the formula $f\,[X_{A1}, X_{A2}, X_F] = (X_{A2} \Rightarrow \neg X_F)$ entailed?

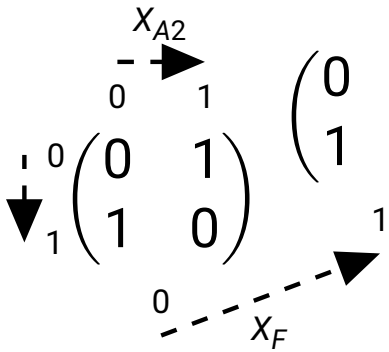
1. Negate the formula f
2. Contract with $\mathcal{KB}$

$$\neg(X_{A2} \Rightarrow \neg X_F) \;=\; \begin{array}{c} \\ x_{A1} \\ \\ \end{array}\!\!\begin{array}{c} x_{A2} \\ \begin{array}{cc} 0 & 1 \end{array} \\ \begin{array}{c} 0 \\ 1 \end{array}\begin{pmatrix} 0 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} \\ \begin{array}{cc} 0 & 1 \end{array} \\ x_F \end{array}\!\!\begin{array}{c} \\ \\ \\ \end{array} [X_{A1}, X_{A2}, X_F]$$

Entailment in the toy accounting system

Recall: The toy accounting system $\mathcal{KB}\,[X_{A1}, X_{A2}, X_F]$ is represented by the tensor

$$(X_{A1} \oplus X_{A2}) \wedge (X_F \Rightarrow X_{A1}) \;=$$

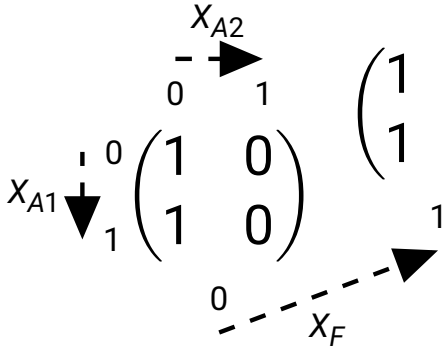


$$\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} [X_{A1}, X_{A2}, X_F]$$

Question: Is the formula $f\,[X_{A1}, X_{A2}, X_F] = (X_{A2} \Rightarrow \neg X_F)$ entailed?

Answer: Since $\langle \mathcal{KB}, \neg f \rangle_{[\varnothing]} \neq 0$ we do not have entailment ($\mathcal{KB} \not\models f$)!

$$(X_{A2} \Rightarrow \neg X_F) \;=$$

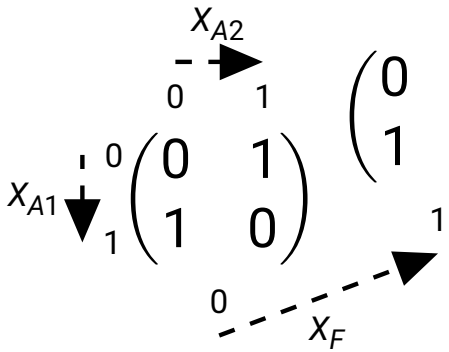


$$\begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} [X_{A1}, X_{A2}, X_F]$$

Entailment in the toy accounting system

Recall: The toy accounting system $\mathcal{KB}\,[X_{A1}, X_{A2}, X_F]$ is represented by the tensor

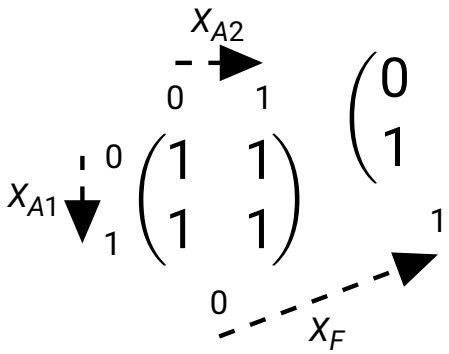
$$(X_{A1} \oplus X_{A2}) \wedge (X_F \Rightarrow X_{A1}) \;=$$



$$\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} [X_{A1}, X_{A2}, X_F]$$

Question: Is the formula $f\,[X_{A1}, X_{A2}, X_F] = \neg(X_F \wedge \neg X_{A1})$ entailed?

$$\neg(X_F \wedge \neg X_{A1}) \;=$$

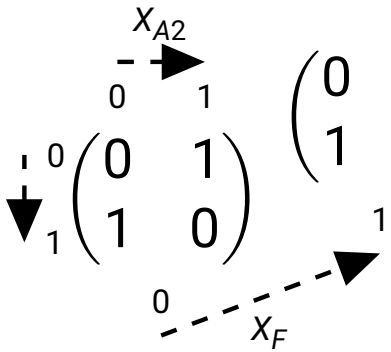


$$\begin{pmatrix} 0 & 0 \\ 1 & 1 \end{pmatrix} [X_{A1}, X_{A2}, X_F]$$

Entailment in the toy accounting system

Recall: The toy accounting system $\mathcal{KB}\,[X_{A1}, X_{A2}, X_F]$ is represented by the tensor

$$(X_{A1} \oplus X_{A2}) \wedge (X_F \Rightarrow X_{A1}) \;=$$

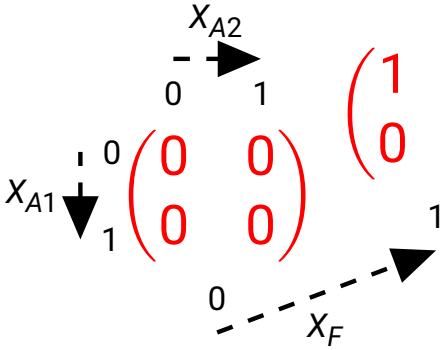


$$\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} [X_{A1}, X_{A2}, X_F]$$

Question: Is the formula $f\,[X_{A1}, X_{A2}, X_F] = \neg(X_F \wedge \neg X_{A1})$ entailed?

1. Negate the formula f

$$\neg\neg(X_F \wedge \neg X_{A1}) \;=$$

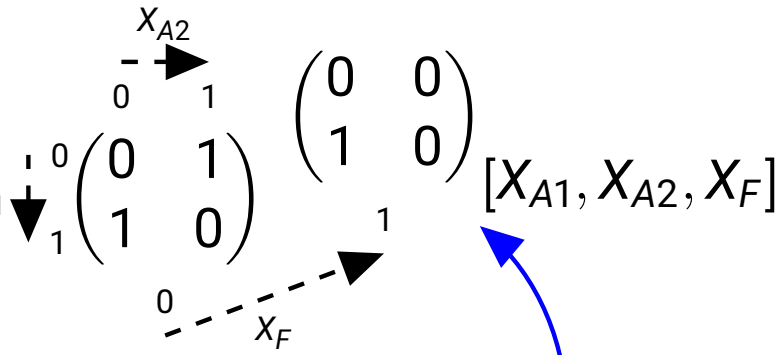


$$\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} [X_{A1}, X_{A2}, X_F]$$

Entailment in the toy accounting system

Recall: The toy accounting system $\mathcal{KB}\,[X_{A1}, X_{A2}, X_F]$ is represented by the tensor

$$(X_{A1} \oplus X_{A2}) \wedge (X_F \Rightarrow X_{A1}) \;=$$



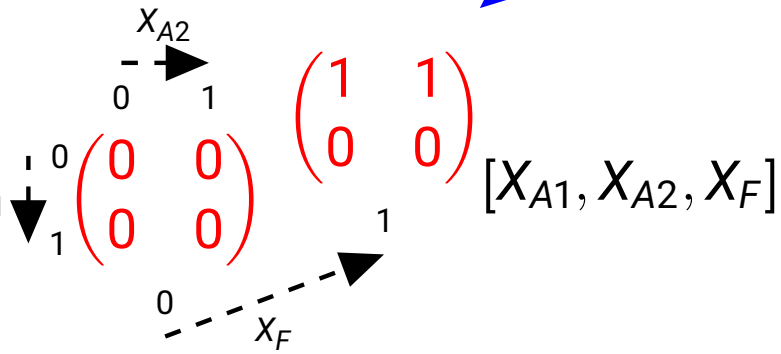
$$\begin{pmatrix} 0 & 0 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} [X_{A1}, X_{A2}, X_F]$$

$$\langle \mathcal{KB}, \neg f \rangle_{[\varnothing]} = 0$$

Question: Is the formula $f\,[X_{A1}, X_{A2}, X_F] = \neg(X_F \wedge \neg X_{A1})$ entailed?

1. Negate the formula f
2. Contract with $\mathcal{KB}$

$$\neg\neg(X_F \wedge \neg X_{A1}) \;=$$

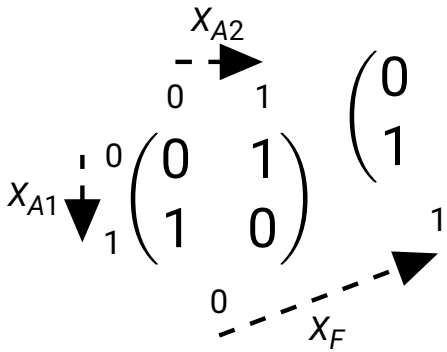


$$\begin{pmatrix} 1 & 1 \\ 0 & 0 \end{pmatrix} \begin{pmatrix} 0 & 0 \\ 0 & 0 \end{pmatrix} [X_{A1}, X_{A2}, X_F]$$

Entailment in the toy accounting system

Recall: The toy accounting system $\mathcal{KB}\,[X_{A1}, X_{A2}, X_F]$ is represented by the tensor

$$(X_{A1} \oplus X_{A2}) \wedge (X_F \Rightarrow X_{A1}) \;=$$

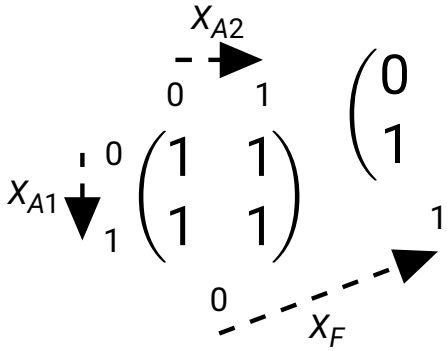


$$[X_{A1}, X_{A2}, X_F]$$

Question: Is the formula $f\,[X_{A1}, X_{A2}, X_F] = \neg(X_F \wedge \neg X_{A1})$ entailed?

Answer: Since $\langle \mathcal{KB}, \neg f \rangle_{[\varnothing]} = 0$ we have entailment ($\mathcal{KB} \models f$)!

$$\neg(X_F \wedge \neg X_{A1}) \;=$$



$$[X_{A1}, X_{A2}, X_F]$$

Outline

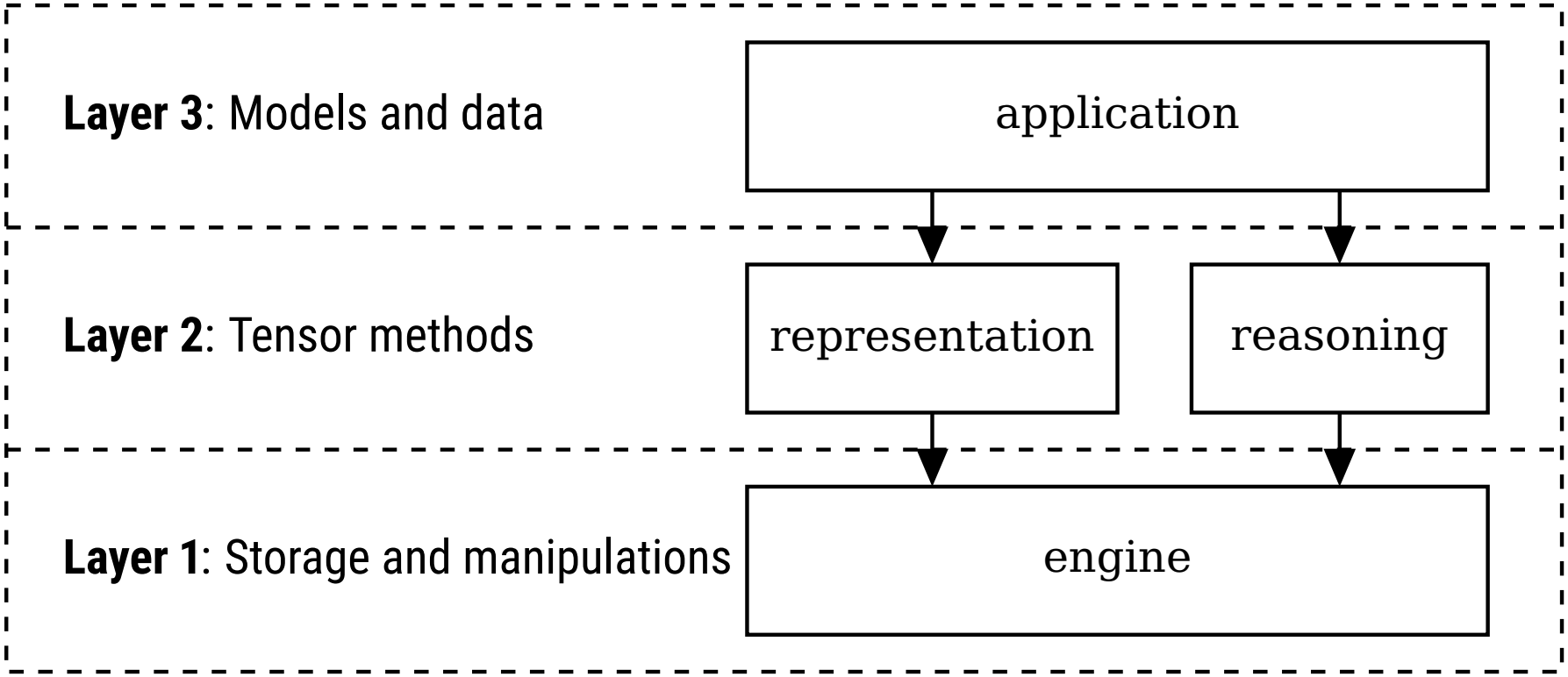
- Knowledge Bases
- Tensor Networks
- Contractions
- Entailment
- Python package `tnreason`

Python package tnreason

tnreason = Tensor-Network Reasoning

Functionality:

- Representations of logical and probabilistic models as tensor networks
- Neuro-symbolic reasoning based on tensor network contractions



Installation

tnreason is maintained on github:

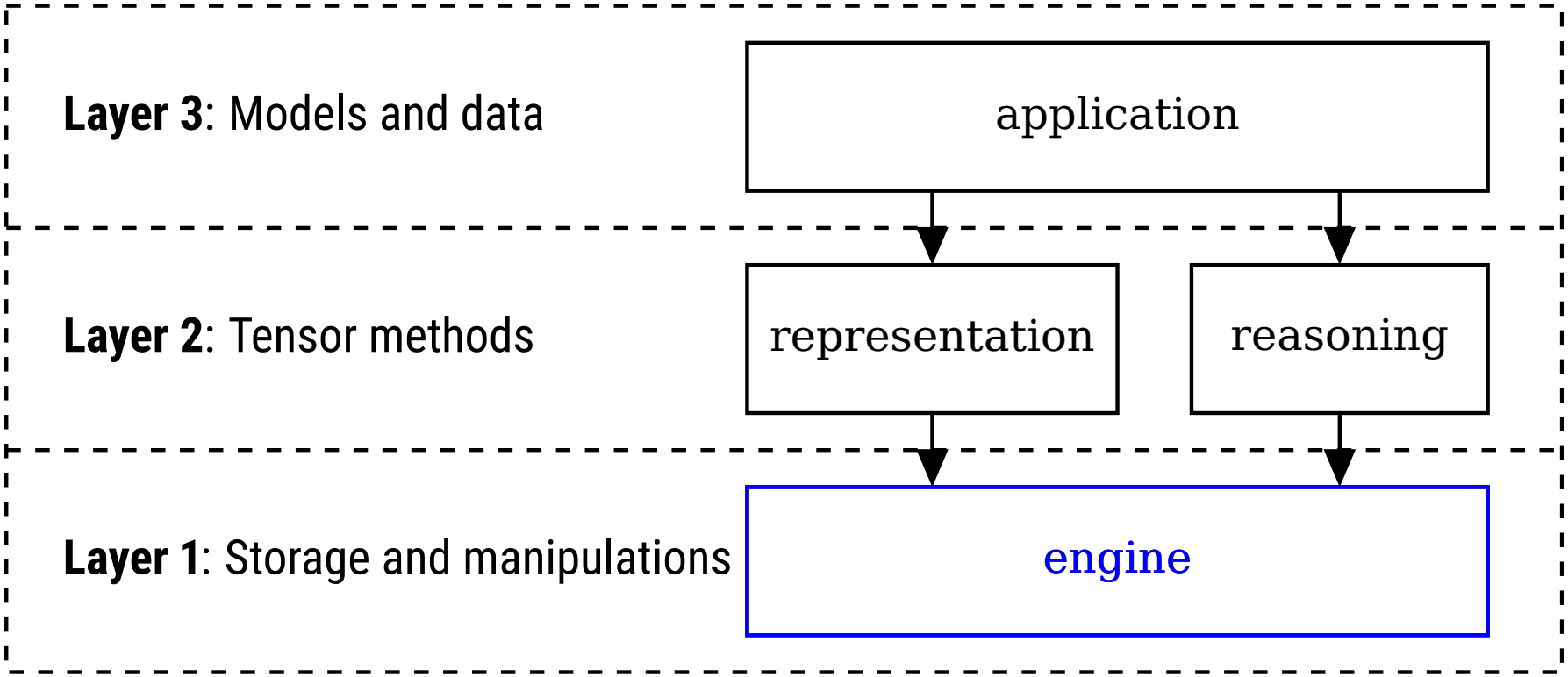
https://github.com/tnreason/tnreason-py

Installation using pip:

```
pip install tnreason==2.0.0
```

Implementation in tnreason: Subpackage engine

The subpackage engineis dedicated to tensor networks and contractions.



Subpackage engine: Tensors

The class `engine.TensorCore`

- colors (list of str): stores strings representing X_k for $k \in [d]$
- shape (list of int): stores the leg dimensions m_k for $k \in [d]$
- name (str): to distinguish from other cores
- values (array-like): stores the coordinates (differing in sub-classes)

The implemented core types differ in the values argument. Cores are instantiated by

```
from tnreason import engine
tensorCore = engine.getCore(coreType)(coreValues,
↪  coreColors, coreName)
```

Subpackage engine: Tensor networks

Tensor networks $\tau^{\mathcal{G}}$ in $\mathrm{tnreason}$ are dictionaries `coreDict` of str tensor nemes valued by `engine.TensorCore` instances.

The hypergraph $\mathcal{G} = (\mathcal{E}, \mathcal{V})$ is stored

- Each e is represented by an instance `tensorCore` of `engine.TensorCore`
- The incidence relations are stored by `tensorCore.values`

Visualization with `matplotlib`:

```
engine.draw_fractor_graph(coreDict)
```

Subpackage engine: Contractions

Reflected in the notation

$$\left\langle \tau^{\mathcal{G}} \right\rangle_{[X_{\mathcal{U}}]}$$

contraction calls are done by

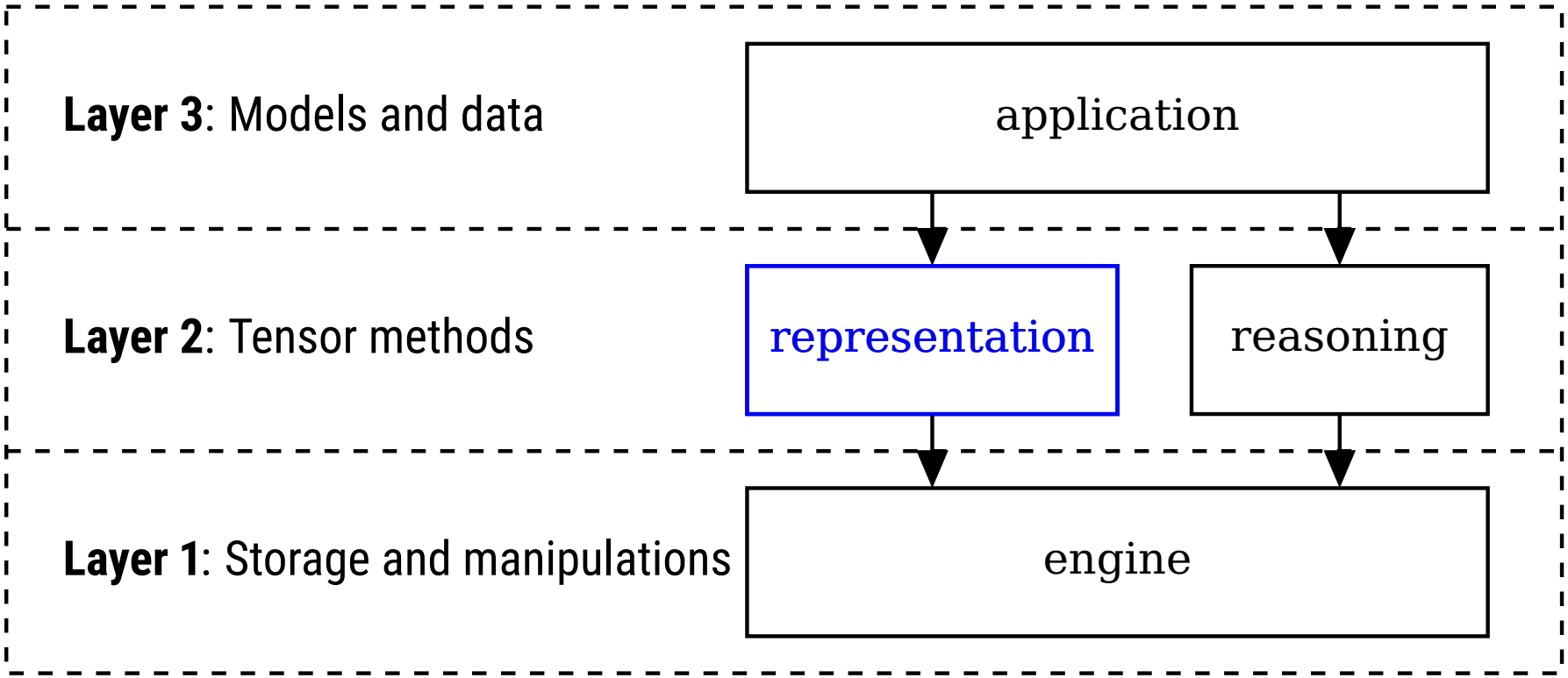
```
from tnreason import engine
contractedNetwork = engine.contract(coreDict,
↪  openColors)
```

Where

- coreDict: Dictionary of tensor cores representing $\tau^{\mathcal{G}}$
- openColors: List of str identifying the variables (and their order) $X_{\mathcal{U}}$ left open in the contraction

Implementations in tnreason**:** **Subpackage** representation

The subpackage representation is dedicated to tensor encoding schemes.



Subpackage representation**: Coordinate encodings**

Creation of a tensor $\tau\left[X_{[d]}\right]$ when having a function computing each coordinate:

```
from tnreason import representation
tensorCore = representation.create_tensor_encoding(
    inShape, inColors, outColors, function)
```

- inShape, inColors: specifying the leg dimension and the names of the variables ($X_{[d]}$)
- function (e.g. in lambda calculus): computes the coordinate given the state of the variables